THEADORO

DOCUMENTAÇÃO DO PRODUTO

THEADORO

Marketplace de presentes

Como a plataforma funciona, o que já está pronto, quais integrações estão ativas e o que a equipe precisa fazer para colocá-la no ar.

VERSÃO	ATUALIZADO EM	ESCOPO
1.0	novembro de 2026	Produto · Operação · Tecnologia

EM UMA PÁGINA

Um marketplace de presentes, com três papéis e entrega local

A THEADORO conecta quem quer presentear com vendedores de presentes, produtos físicos e serviços agendáveis. A operação acontece em um só lugar — catálogo, pedidos, pagamentos e entrega — e a entrega é feita na região de Cuiabá/MT.

Comprador

Navega, monta o carrinho, paga com cartão, PIX ou boleto, acompanha o pedido e guarda favoritos.

Vendedor

Cadastra produtos, gerencia vendas, cupons, campanhas e a própria assinatura na plataforma.

Administrador

Aprova vendedores, cuida do catálogo, dos pedidos, do conteúdo e das integrações.

INTEGRAÇÕES	Pagar.me (pagamentos), Uber Direct (entrega), Gemini (sugestão de presentes com IA) e ViaCEP (endereços).
PAGAMENTO	Cartão de crédito, PIX e boleto. A confirmação chega automaticamente ao sistema, sem intervenção manual.
ENTREGA	Cotação e despacho reais pela Uber Direct, restritos a Cuiabá/MT. O código de rastreio fica no pedido.
ADMINISTRAÇÃO	Um painel único para usuários, vendedores, pedidos, catálogo, relatórios e configurações.
ESCOPO	Segue o README do projeto. As seções finais são técnicas e servem à equipe que for manter o sistema.

NESTE DOCUMENTO

Sumário

01 Visão do produto

02 Stack

03 Arquitetura

04 Estrutura do repositório

05 Requisitos

06 Configuração do ambiente

07 Rodando localmente

08 Banco de dados

09 Edge Functions

10 Integrações

11 Fluxos principais

12 Build e deploy

13 Verificação

14 Limitações conhecidas

15 Checklist de go-live

Visão do produto

Papel	Entra por	O que faz
Comprador	`/auth` → `/buyer/*`	navega, monta carrinho, finaliza compra, acompanha pedidos, favoritos e listas de presentes
Vendedor	`/seller/onboarding` → `/seller/*`	cadastra produtos, gerencia vendas, cupons, campanhas, assinatura e relatórios
Administrador	`/admin/*`	aprova vendedores, gerencia catálogo, pedidos, conteúdo, integrações e configurações

Diferenciais do produto: curadoria de presentes, experiências agendáveis, assistente de IA para sugestão e oferta/frete como argumento comercial.

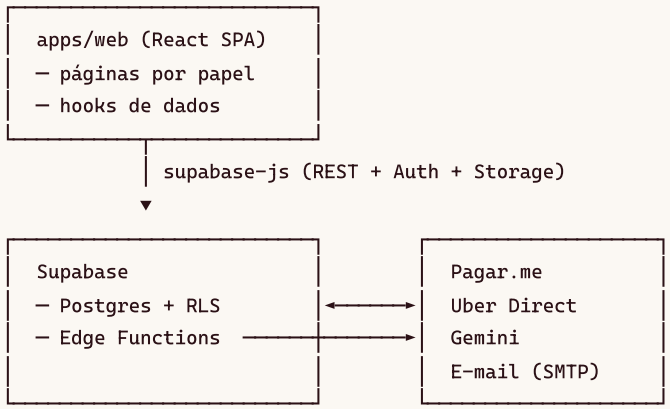
A entrega é **restrita a Cuiabá/MT** — o gate de CEP aceita apenas faixas `780xx` e `781xx` (`apps/web/src/services/shippingService.ts`).

Stack

Camada	Tecnologia
Monorepo	npm workspaces + Turborepo
Frontend	React 18 + Vite 8 + React Router 7 + TanStack Query
UI	Tailwind CSS + shadcn/ui (Radix) — design system em `packages/ui`
Backend / dados	**Supabase** (Postgres, Auth, Storage, Edge Functions)
Pagamentos	Pagar.me v5 (cartão, PIX, boleto)
Entrega	Uber Direct (cotação e despacho)
IA	Google Gemini via edge function `ai-gift-assistant`
Cliente de API tipado	`packages/api-client` (gerado a partir do `swagger.json` com Kubb)
Deploy do front	Node.js na Hostinger (`server.cjs` serve o build estático)

Idioma do produto: português do Brasil. **Moeda:** BRL.

Arquitetura



Regras que a arquitetura impõe:

1. **Nenhum segredo no browser.** Chaves privadas (Pagar.me `sk_live`, Uber `client_secret`, autenticação do webhook) vivem em edge functions — nunca no bundle.
2. **Toda escrita sensível passa por policy de RLS** ou por edge function com `service_role`.
3. **O front não chama gateway direto.** Ele chama `supabase.functions.invoke(...)`, e a function fala com o provedor.
4. `packages/ui` **é compartilhado** entre páginas, mas não contém lógica de negócio.

Estrutura do repositório

```
theadoro-gift/
├── apps/
│   └── web/                  # SPA React + Vite
│       ├── public/           # ícones, robots.txt, .htaccess (fallback de SPA)
│       └── src/
│           ├── components/   # UI por domínio (auth, checkout, payments, layouts…)
│           ├── contexts/     # Auth, Cart, Wishlist
│           ├── hooks/        # dados (useProducts, useOrders, useCheckout…)
│           ├── integrations/supabase/   # client + tipos do banco
│           ├── lib/          # utilitários
│           ├── pages/        # rotas (públicas, admin, buyer, seller)
│           ├── schemas/      # validação (zod)
│           └── services/     # payment, shipping, pix, cep
├── packages/
│   ├── api-client/           # cliente tipado gerado (Kubb) a partir de swagger.json
│   ├── config/               # tailwind + typescript compartilhados
│   └── ui/                   # design system (shadcn)
├── supabase/
│   ├── config.toml           # configuração do projeto + verify_jwt por function
│   ├── migrations/           # baseline + bootstrap + seed (histórico em migrations/archive/)
│   └── functions/            # 7 edge functions (Deno)
├── scripts/                  # utilitários legados de reparo (não fazem parte do runtime)
├── server.cjs                # servidor HTTP de produção (serve /dist)
├── start.cjs                 # entrypoint de deploy: build + server
└── turbo.json
```

> `scripts/` contém scripts one-off usados durante a construção do produto (checagem de colunas, reparo de perfis, sync de vendedores). Eles chamam `exec_sql` com a service role e **não são necessários para rodar o produto**.

Requisitos

- **Node.js 18+** (recomendado 20 LTS)
- **npm 10** (o projeto usa workspaces)
- **Supabase CLI** — `npm i -g supabase` ou instalador oficial
- Acesso ao projeto Supabase `pfpbaepzvgovqheeqxlg` (organização TheAdoro)

Configuração do ambiente

Copie o exemplo e preencha:

```
cp .env.example apps/web/.env.local
```

Variáveis do frontend (build-time, entram no bundle)

Variável	Onde	Descrição
`VITE_SUPABASE_URL`	`apps/web/.env.local` e `.env.production`	URL do projeto: `https://pfpbaepzvgovqheeqxlg.supabase.co`
`VITE_SUPABASE_PUBLISHABLE_KEY`	idem	chave pública (`anon` /publishable). **É pública por design**
`VITE_SUPABASE_PROJECT_ID`	`.env.production`	`pfpbaepzvgovqheeqxlg`
`VITE_APP_NAME` , `VITE_APP_VERSION` , `VITE_APP_ENV`	`.env.production`	metadados exibidos na interface

Variáveis só de servidor

Variável	Onde	Uso
`SUPABASE_SERVICE_ROLE_KEY`	`.env` local (nunca no bundle)	scripts administrativos e chamadas server-side
`UBER_CLIENT_ID` , `UBER_CLIENT_SECRET` , `UBER_CUSTOMER_ID`	secrets da function	Uber Direct
`PAGARME_WEBHOOK_USER` + `PAGARME_WEBHOOK_PASSWORD`	secrets da function	Basic Auth do webhook
`PAGARME_WEBHOOK_SECRET`	alternativa	header `x-webhook-secret`

> ⚠️ `.env` e `.env.production` estão no `.gitignore` . **Nunca** comite chaves privadas. As chaves do Pagar.me (`pk_*` / `sk_*`) **não** ficam em env: são cadastradas em **Admin → Configurações → Pagamentos** e lidas pela edge function `process-payment` a partir de `site_settings` .

Rodando localmente

```
npm install          # instala o monorepo
npm run dev          # turbo dev → apps/web em http://localhost:5173
```

Outros comandos úteis:

```
npm run build        # turbo build + scripts/sync-dist.cjs → /dist na raiz
npm run lint         # eslint em todos os workspaces
npm start            # produção: build + server.cjs
npx vite build --mode development   # build de desenvolvimento
```

Banco de dados

Projeto: `pfpbaepzvgovqheeqxlg` (região `sa-east-1`, Postgres 17).

Migrations

- `supabase/migrations/20261109000000_baseline_public_schema.sql` é a **fonte de verdade do schema**: enums, tabelas, constraints, índices, views, functions, triggers, RLS, policies e Storage, gerados do banco de produção em 2026-11-09.
- É idempotente e serve para reconstruir um projeto novo do zero.
- As migrations antigas estão em `supabase/migrations/archive/` somente como histórico; **não as aplique** em um projeto novo.
- Toda tabela nova **precisa de RLS habilitado** e policies explícitas.

Projeto novo:

```
supabase link --project-ref <novo-ref>
supabase db push
```

Depois do push, rode também o bootstrap do admin master e o seed do catálogo (as duas migrations já estão em `supabase/migrations/`).

> No projeto de produção atual o schema já corresponde ao baseline. Evite `db push` cego nele: o histórico remoto das migrations antigas não bate com os arquivos locais. Mudanças novas entram como migration normal.

Admin master

A migration `20261109000001_bootstrap_master_admin.sql` (roda depois do baseline) cria o administrador principal de uma instalação nova:

Campo	Valor padrão
E-mail	`admin@theadoro.com.br`
Senha	`theadoro@2025`
Papel	`admin` em `profiles.role` **e** em `user_roles`
Situação	e-mail confirmado, `verified` e `is_approved`

A migration é **idempotente**: se o e-mail já existir, ela não mexe na senha e apenas garante o papel de admin e o perfil. Rode-a junto com `supabase db push` ou cole o SQL no **SQL Editor**.

```
supabase link --project-ref pfpbaepzvgovqheeqxlg
supabase db push        # aplica, entre outras, a migration do admin master
```

> ⚠️ **Troque a senha no primeiro acesso** (Admin → Usuários → *Alterar Senha*) ou edite a constante `v_password` da migration antes de aplicá-la em produção.

Tabelas (29) e views (2)

Domínio	Tabelas
Catálogo	`products`, `product_variations`, `categories`, `reviews`, `wishlist`, `cart`, `product_analytics`
Pedidos	`orders`, `order_items`, `payments`, `payment_events`
Pessoas	`profiles`, `user_roles`, `user_addresses`, `seller_applications`
Vendedor	`seller_subscriptions`, `subscription_plans`, `seller_invoices`, `campaigns`, `campaign_participants`, `coupons`
Serviços	`appointments`, `delivery_slots`, `service_availability`
Presentes	`gift_wraps`, `gift_lists`, `gift_list_items`, `important_dates`
Plataforma	`site_settings`

Views: `seller_summary` (perfil público do vendedor) e `product_stats_daily` (métricas por dia).

`orders` guarda os campos de entrega: `shipping_method`, `shipping_address` (JSON), `shipping_provider`, `shipping_quote_id`, `uber_delivery_id`, `tracking_url`.

Funções e triggers

- `handle_new_user` — cria perfil no signup (trigger em `auth.users`).
- `handle_updated_at` / `update_updated_at_column` — timestamps.
- `handle_seller_approval_subscription` — efeitos da aprovação de vendedor.
- `handle_default_address` — garante um único endereço padrão.
- `generate_product_slug` / `slugify` — URLs.

- `decrement_product_stock` / `restore_product_stock` — baixa e devolução atômica de estoque no checkout.
- `increment_coupon_usage` — contador de uso do cupom.
- Helpers de autorização usados por RLS: `has_role`, `is_admin`, `can_access_order`, `is_order_seller_check`, `get_my_roles`, `get_my_profile_v2`.

> **Segurança do banco:** `public.exec_sql(query text)` é `SECURITY DEFINER` e executável por `anon` — qualquer visitante com a chave pública podia rodar SQL arbitrário. O `EXECUTE` foi revogado de `PUBLIC`, `anon` e `authenticated`, mantido apenas para `service_role`. O mesmo foi feito para os helpers de introspecção (`get_table_columns`, `get_table_info`, `list_cols`, `get_coupons_columns`, `get_user_debug_info`). Se algum script de `scripts/` parar de funcionar, ele precisa da `service_role` key — e isso é o comportamento correto.

Edge Functions

Function	verify_jwt	Responsabilidade
`process-payment`	`true`	cria cobranças do pedido no Pagar.me (cartão, PIX, boleto), consulta e cancela
`seller-invoice-payment`	`false`	cobra a fatura de assinatura do vendedor (cartão/PIX) e troca/cancela plano
`pagarme-webhook`	`false`	recebe eventos do Pagar.me, confirma pedidos **e faturas**, dispara o despacho
`send-email`	`false`	e-mail transacional (SMTP ou Resend)
`uber-delivery`	`true`	cotação (`action: quote`) e despacho (`action: request`) na Uber Direct
`admin-users`	`false`	exclusão de usuário (com conferência de dependências) e troca de e-mail/senha pelo admin
`ai-gift-assistant`	padrão (`true`)	sugestão de presentes com Gemini

`verify_jwt = false` não significa "aberta": cada function com essa configuração valida o JWT ou a credencial por conta própria. `process-payment` e `uber-delivery` exigem usuário logado porque o gateway aceita a chave anônima como JWT válido — a checagem é refeita dentro da function. `pagarme-webhook` exige Basic Auth/secrets; `seller-invoice-payment` valida o usuário e o vínculo da fatura; `send-email` valida o usuário para não virar relay.

Secrets

```
supabase secrets set UBER_CLIENT_ID=... UBER_CLIENT_SECRET=... UBER_CUSTOMER_ID=...
supabase secrets set PAGARME_WEBHOOK_USER=... PAGARME_WEBHOOK_PASSWORD=...
# e-mail (escolha SMTP ou Resend)
supabase secrets set EMAIL_FROM="TheAdoro <noreply@theadoro.com.br>"
supabase secrets set SMTP_HOST=smtp.hostinger.com SMTP_PORT=465 SMTP_SECURE=true \
  SMTP_USER=noreply@theadoro.com.br SMTP_PASS=...
# ou:
supabase secrets set RESEND_API_KEY=...
```

`SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY` já existem no runtime de toda function.

Deploy

```
supabase functions deploy process-payment          --project-ref pfpbaepzvgovqheeqxlg
supabase functions deploy seller-invoice-payment   --project-ref pfpbaepzvgovqheeqxlg --no-verify-jwt
supabase functions deploy pagarme-webhook          --project-ref pfpbaepzvgovqheeqxlg --no-verify-jwt
supabase functions deploy send-email               --project-ref pfpbaepzvgovqheeqxlg --no-verify-jwt
supabase functions deploy uber-delivery            --project-ref pfpbaepzvgovqheeqxlg
supabase functions deploy admin-users              --project-ref pfpbaepzvgovqheeqxlg --no-verify-jwt
supabase functions deploy ai-gift-assistant        --project-ref pfpbaepzvgovqheeqxlg
```

Desenvolvimento local

```
supabase functions serve uber-delivery --env-file supabase/.env.local
```

Integrações

Pagar.me — pagamentos

1. **Tokenização no browser** — `services/payment.service.ts` chama `POST https://api.pagar.me/core/v5/tokens` com a **public key** cadastrada no admin. O número do cartão nunca passa pelo backend.
2. **Criação da cobrança no servidor** — `process-payment` monta cliente, endereço, itens e frete e chama `POST /core/v5/orders`. Suporta PIX (`pix_code`, `pix_url`, expiração), cartão e boleto.
3. **Confirmação** — `pagarme-webhook` recebe `charge.paid` / `order.paid`, marca `payments.status = approved`, move o pedido para `processing` e, se a entrega for Uber, solicita a corrida.

Cadastre no painel do Pagar.me:

```
URL: https://pfpbaepzvgovqheeqxlg.supabase.co/functions/v1/pagarme-webhook
Eventos: charge.paid, order.paid, charge.payment_failed, order.payment_failed
Autenticação: Basic Auth (mesmo usuário/senha dos secrets)
```

O webhook é **idempotente**: evento repetido com pagamento já aprovado não reprocessa a transição, mas confere o despacho.

Uber Direct — entrega

Referência: https://developer.uber.com/docs/deliveries/get-started

```
OAuth:      POST https://auth.uber.com/oauth/v2/token   (client_credentials, scope eats.deliveries)
Cotação:    POST https://api.uber.com/v1/customers/{customer_id}/delivery_quotes
Despacho:   POST https://api.uber.com/v1/customers/{customer_id}/deliveries
```

- Credenciais: secrets `UBER_*` **ou** os campos em **Admin → Configurações → Integrações** (os secrets têm precedência). O valor é lido exatamente como salvo.
- Endereços vão como string JSON (`street_address[]`, `city`, `state`, `zip_code`, `country: "BR"`). CEP sem logradouro é resolvido via ViaCEP dentro da function.
- **Coleta:** `store_address`, `store_city`, `store_state`, `store_cep` e `store_phone` (o telefone é obrigatório para a Uber).
- **Despacho automático:** cartão aprovado no checkout. **PIX/boleto:** o webhook dispara; se o webhook ainda não estiver configurado, o botão **"Despachar Uber"** em Admin → Pedidos faz o mesmo.
- O despacho é **idempotente** por `orders.uber_delivery_id` — nunca gera duas corridas.

Correios — ⚠️ simulado

`shippingService.calculateCorreios()` retorna preço calculado localmente (`28.90 + peso × 4`), **sem chamar nenhuma API**. Serve para o fluxo fechar, não para cobrar frete real.

tIAdoro (IA)

`ai-gift-assistant` recebe a descrição do presente e responde com sugestões usando Gemini. Requer a chave do provedor configurada nos secrets.

Fluxos principais

Checkout

```
carrinho → CEP → cotação de frete → dados/CPF → revalidação de preço e estoque → pedido pendente → itens → baixa de estoque → cobrança → pagamento persistido → (despacho) → agendamentos
```

`apps/web/src/hooks/useCheckout.ts` orquestra tudo. A ordem importa: o **pedido e os itens são criados antes da cobrança**. Se o gateway recusar ou algo falhar, o pedido é cancelado e o estoque devolvido — o cliente nunca fica pago e sem pedido. `merchant_order_code` (`TA-…`) é o elo entre Pagar.me e o pedido, e o cupom é relido do banco antes de compor o total.

Pagamento

- **Cartão:** aprovado na hora → pedido nasce `processing` → despacho Uber imediato.
- **PIX/boleto:** pedido nasce `pending` → webhook confirma → `processing` → despacho.
- O webhook já está deployado e autenticado; falta **cadastrar a URL no painel do Pagar.me**. Sem esse cadastro, PIX fica `pending` até o pagamento ser confirmado manualmente pelo admin.

Frete e despacho

1. `calculateShipping(cep, items, sellerId)` valida a região (Cuiabá), busca configurações e consulta Uber e Correios em paralelo.
2. Uber devolve `fee` real; a tarifa percentual de `shipping_express_tariff` é aplicada como **markup** sobre esse valor.
3. O `quote_id` da Uber é gravado em `orders.shipping_quote_id` — sem ele, o despacho é recusado (`quote_required`).

Build e deploy

Frontend — Hostinger (aplicação Node)

O projeto é publicado como **app Node** na Hostinger, não como site estático.

Campo	Valor
Application root	raiz do repositório
Node.js	20.x
Build command	`npm run build`
Output directory	`dist`
Startup file	`start.cjs`

- `start.cjs` instala devDependencies se faltarem, roda o build, valida `dist/index.html` e sobe o servidor. Isso existe porque a Hostinger executa o startup file com `node`, não com `npm start`.
- `server.cjs` serve `/dist` com fallback de SPA, `Cache-Control` imutável em `/assets`, `no-cache` no `index.html` e bloqueio de path traversal.
- `apps/web/public/.htaccess` (fallback de SPA) também é copiado para o build, para o caso de hospedagem Apache/LiteSpeed.

CI — GitHub Actions

O deploy das functions pode ser automatizado por `.github/workflows/deploy-supabase-functions.yml` (push que toque `supabase/**` ou `supabase/config.toml`, além de `workflow_dispatch`).

Requer dois secrets no repositório: `SUPABASE_ACCESS_TOKEN` e `SUPABASE_PROJECT_ID`.

O caminho canônico de deploy — e o único que não depende de CI — é a CLI:

```
supabase functions deploy <nome> --project-ref pfpbaepzvgovqheeqxlg
```

`verify_jwt` vem do `supabase/config.toml`. Só acrescente `--no-verify-jwt` nas functions marcadas com `false` ali (`pagarme-webhook`, `seller-invoice-payment`, `send-email`, `admin-users`); em `process-payment` e `uber-delivery` a flag desligaria a exigência de login.

Verificação

```
npm run build                                    # termina com "Successfully synced build files to root
/dist"
npm run lint                                     # eslint em todos os workspaces
cd apps/web && npx tsc --noEmit -p tsconfig.app.json   # 0 erros
cd apps/web && npx vitest run                    # testes unitários
npm audit                                        # 0 vulnerabilidades
```

Sondas rápidas das functions publicadas:

```
curl -s -X POST https://pfpbaepzvgovqheeqxlg.supabase.co/functions/v1/uber-delivery \
  -H "apikey: <anon>" -H "Content-Type: application/json" -d '{"action":"quote"}'
# esperado sem credenciais: 503 {"error":"not_configured"}

curl -s -X POST https://pfpbaepzvgovqheeqxlg.supabase.co/functions/v1/pagarme-webhook \
  -H "apikey: <anon>" -H "Content-Type: application/json" -d '{}'
# esperado sem secrets: 401 {"error":"unauthorized"}

curl -s -X POST https://pfpbaepzvgovqheeqxlg.supabase.co/functions/v1/admin-users \
  -H "Content-Type: application/json" -d '{"action":"preview-delete","userId":"x"}'
# esperado sem JWT de admin: 401 {"error":"forbidden"}
```

Limitações conhecidas

1. **Correios é simulado** — `shippingService.calculateCorreios()` calcula o preço localmente (`28.90 + peso × 4`), sem chamar nenhuma API. Hoje está desligado em produção.
2. **Webhook do Pagar.me precisa ser cadastrado no painel** — os secrets já estão definidos e a function aceita a autenticação; a URL completa está em `supabase/.env.webhook.local`. Sem o cadastro, PIX de pedido e de fatura não confirmam automaticamente.
3. **Compra exige conta** — `/checkout` é rota protegida. Visitante é levado ao login antes de pagar, porque o pedido precisa de `buyer_id`.
4. **Região limitada a Cuiabá/MT** — gate fixo no `shippingService` (faixas `780xx` / `781xx`). Entrega própria do produto continua disponível.
5. **Sem geocodificação** — a Uber recebe o endereço textual, não lat/long. Se a conta exigir coordenadas, a cotação é recusada.
6. **E-mail transacional depende de credencial** — a function `send-email` já existe e o `EmailService` chama de verdade (SMTP Hostinger ou Resend). Enquanto `SMTP_PASS` / `RESEND_API_KEY` não for definido, ela responde 503 e o checkout segue normalmente, sem e-mail.
7. `packages/api-client` depende de `swagger.json`; regenerar exige migrar o `kubb.config.ts` (o plugin `@kubb/plugin-tanstack-query` foi removido do npm). O `dist/` já construído atende o app.

Checklist de go-live

- [ ] `.env.local` criado a partir de `.env.example`
- [ ] `npm install`, `npm run build` e `npm run dev` funcionando
- [ ] `supabase link` no projeto `pfpbaepzvgovqheeqxlg`
- [ ] Baseline `20261109000000_baseline_public_schema.sql` aplicado (reconstrói o schema inteiro)
- [ ] `20261109000001_bootstrap_master_admin.sql` aplicado (cria `admin@theadoro.com.br`)
- [ ] `20261109000002_seed_catalog.sql` aplicado (categorias, planos, embalagens)
- [ ] **Senha do admin master trocada** em Admin → Usuários
- [ ] Secrets `UBER_*`, `PAGARME_WEBHOOK_*` e de e-mail (`SMTP_*` ou `RESEND_API_KEY`) configurados
- [ ] Webhook do Pagar.me cadastrado com a URL de `supabase/.env.webhook.local`
- [ ] `process-payment` e `uber-delivery` deployadas com `verify_jwt = true`
- [ ] Credenciais da Uber preenchidas no admin (sandbox primeiro)
- [ ] `store_address`, `store_city`, `store_state`, `store_cep`, `store_phone` preenchidos
- [ ] Deploy da Hostinger apontando o startup file para `start.cjs`
- [ ] Rotacionar credenciais que já tenham circulado: `service_role` key, PAT do Supabase e senha do banco